

ANTLR 4

ANother Tool for Language Recognition

Daniel Oliveira

UFPR

Agosto 2026

ANTLR 4

- ▶ *ANother Tool for Language Recognition* é uma ferramenta que inclui análise léxica e sintática
 - ▶ Já produzindo uma árvore gramatical após a análise sintática
- ▶ Podemos separar a parte léxica e a gramática do código de implementação
 - ▶ Arquivos com extensão *g4* contém apenas instruções para o analisador léxico e a gramática da linguagem
 - ▶ Embora podemos colocar código diretamente no arquivo

ANTLR 4

- ▶ Com a árvore gramatical feita, podemos percorrê-la usando os *design patterns*:
 - ▶ listener: usado para análises específicas e “pequenas”, que verificam apenas parte da árvore; código pode ser executado ao entrar (entry) ou sair (exit) de um nó
 - ▶ visitor: usado para compilação e tarefas mais complicadas, podemos escolher a forma que a árvore é percorrida (pre, in, post, dfs, ...)
- ▶ ANTLR 4 é feito em Java, mas pode gerar código alvo para linguagens diferentes: Java, C#, Python3, JavaScript, TypeScript, Go, C++, Swift, Dart, PHP
 - ▶ Usaremos C++ para, posteriormente, integrar com LLVM e MLIR

Documentação

- ▶ A documentação para o ANTLR 4 está disponível no link
 - ▶ <https://github.com/antlr/antlr4/blob/master/doc/getting-started.md>
- ▶ E a documentação para C++:
 - ▶ <https://github.com/antlr/antlr4/blob/master/doc/cpp-target.md>

Download e Instalação

- ▶ Primeiro passo, faça o download do jar:
 - ▶ Baixo no site: <https://wwwantlr.org/download.html>
 - ▶ link direto: <https://wwwantlr.org/download/antlr-4.13.2-complete.jar>
 - ▶ `wget https://wwwantlr.org/download/antlr-4.13.2-complete.jar`
 - ▶ `curl -O https://wwwantlr.org/download/antlr-4.13.2-complete.jar`

- ▶ Coloque o *jar* em algum lugar razoável (/path/para/arquivo/), e coloque as seguintes linhas em seu `.bashrc`:

```
export CLASSPATH=".:path/para/arquivo/antlr-4.13.2-complete.jar:$CLASSPATH"
alias antlr4='java -Xmx500M -cp "/path/para/arquivo/antlr-4.13.2-complete.jar:$CLASSPATH" org.antlr.v4.Tool'
alias grun='java -Xmx500M -cp "/path/para/arquivo/antlr-4.13.2-complete.jar:$CLASSPATH" org.antlr.v4.gui.TestRig'
```

C++ Target

- ▶ Para usar o antlr com C++, precisamos informá-lo que deve gerar código em C++:
 - ▶ `antlr4 -Dlanguage=Cpp MyGrammar.g4`
- ▶ Porém usaremos CMake pois facilitará (a princípio) a posterior integração com LLVM e MLIR

Exemplo Simples

- ▶ Um exemplo simples está disponível no repositório da disciplina
 - ▶ https://github.com/dagoliveira/compiladores/tree/main/antlr4/expr_simples
- ▶ Edite o arquivo `CMakeLists.txt` e atualize a linha abaixo para apontar corretamente ao arquivo jar do ANTLR 4:

```
set(ANTLR_EXECUTABLE /usr/local/lib/antlr-4.13.2-complete.jar)
```

Exemplo Simples

- ▶ Entre na pasta `expr_simples` e faça os encantamentos tradicionais do CMake para compilar e gerar o executável `expr` (e espere com paciência):

```
mkdir build; cd build  
cmake ..  
make
```

Exemplo Simples

- Teste o executável, a cada sentença válida a árvore será impressa:

```
$ ./expr
```

```
Entre com um input ('q' para sair): a = 1+2*3
```

```
Sentença ok!
```

```
Árvore sintática:
```

```
(prog (stat a = (expr (expr 1) + (expr (expr 2) * (expr 3))) \n))
```

```
Entre com um input ('q' para sair): a+2*b
```

```
Sentença ok!
```

```
Árvore sintática:
```

```
(prog (stat (expr (expr a) + (expr (expr 2) * (expr b))) \n))
```

```
Entre com um input ('q' para sair): q
```

Exemplo Simples - Gramática

- ▶ Abra o arquivo `LabeledExpr.g4` e verifique a gramática
- ▶ Comandos léxicos estão em letras maiúsculas:
 - ▶ `MUL DIV ADD SUB ID INT NEWLINE WS`
- ▶ A gramática em si utiliza o formato EBNF (Extended Backus–Naur form), com os símbolos não terminais em minúsculo:
 - ▶ `prog stat expr`

Exemplo Simples

- ▶ Mais informações sobre como criar as regras léxicas e sintáticas estão na documentação oficial, além do livro “The Definitive ANTLR 4 Reference”
 - ▶ <https://github.com/antlr/antlr4/blob/master/doc/index.md>
- ▶ Vários exemplos de gramáticas prontas para o ANTLR 4 estão disponíveis no seguinte repositório:
 - ▶ <https://github.com/antlr/grammars-v4>

Exercício

- ▶ Agora vamos refazer o exercício de implementar uma calculadora, agora percorrendo a árvore gerada pelo ANTLR 4
- ▶ Os arquivos base estão no repositório da disciplina (antlr4/expr_exercicio):
 - ▶ https://github.com/dagoliveira/compiladores/tree/main/antlr4/expr_exercicio

Exercício

- ▶ ANTLR 4 vai gerar a classe `LabeledExprBaseVisitor`, com o cmake atual, no arquivo chamado `LabeledExprBaseVisitor.h` em `build/antlr4cpp_generated_src/LabeledExpr/LabeledExprBaseVisitor.h`
 - ▶ Vamos estender essa classe, implementando apenas os métodos que nos interessam
 - ▶ Não precisamos implementar (mudar o default) os métodos `visitProg` e `visitBlank`
- ▶ O arquivo `EvalVisitor.hpp` e `EvalVisiton.cpp` são os arquivos que precisamos modificar

Exercício

- ▶ Podemos resolver esse exercício por meio de duas abordagens:
 - ▶ Usando o retorno de cada função
 - ▶ Decorando a árvore, associando a cada nó um valor (ou valores) diferente

Exercício - valor de retorno

- ▶ Cada função no *visitor pattern* que o ANTLR 4 gera retorna um valor do tipo `std::any`
 - ▶ Ou seja, permite retornar qualquer coisa
- ▶ Dessa forma, podemos retornar um inteiro após visitar cada nó da árvore
 - ▶ Exemplos no arquivo `EvalVisitor.cpp`

Exercício - decorando a árvore

- ▶ Não vamos, exatamente, decorá-la
 - ▶ Porém, faremos algo com o mesmo efeito

- ▶ Podemos criar uma hash (map) que associa a cada nó da árvore um valor específico

Exercício - decorando a árvore

- ▶ Iniciamos uma variável usando `antlr4::tree::ParseTreeProperty` e associando algum tipo de dado (como `int`, ou `any`):

```
antlr4::tree::ParseTreeProperty<int> mapa;
```

- ▶ Depois podemos inserir ou recuperar valores associados a cada nó da árvore (`ctx`):

```
int res = ...  
mapa.put(ctx, res);  
...  
int expr_valor = mapa.get(ctx->expr(0));
```